

Angular & .NET Cheatsheet

2026-03-19

Table of Contents

1. Frontend	1
1.1. Authentication	1
1.1.1. Interceptor	1
1.1.2. Auth Guard	1
1.2. Environment	2
1.2.1. Environment Definition	2
1.2.2. Environment Usage	2
1.3. Directives	2
1.3.1. Filter Directives	3
1.3.2. Async Directives	3
1.4. Routing	4
1.4.1. Routes	4
1.4.2. Navigation	5
1.4.3. Path Parameter	5
1.4.4. Query Parameter	6
1.5. Forms	6
1.5.1. Reactive Forms	6
1.5.2. Template Driven Forms	7
1.6. Signals	8
1.6.1. Models	8
1.6.2. Input	9
1.6.3. Output	10
1.7. HTTP Client	11
1.8. Observables	12
1.8.1. subscribe mit next + error	12
1.8.2. Eigenes Observable (Minimalbeispiel)	12
1.9. HTTP Requests	12
1.9.1. GET	12
1.9.2. POST	13
1.9.3. PUT	13
1.9.4. PATCH	13
1.9.5. DELETE	13
1.9.6. Request Parameters	13
1.10. Template Syntax	14
1.10.1. @for	14
1.10.2. @if	15
1.11. Table	15
2. Backend	17

2.1. CDI (Dependency Injection)	17
2.2. Swagger	17
2.2.1. Swagger Builder	17
2.2.2. Cors	18
2.3. Minimal API	18
2.3.1. GET	18
2.3.2. POST	19
2.3.3. PUT	19
2.3.4. PATCH	19
2.3.5. DELETE	20
2.4. Authentication	20
2.5. Services	21
2.5.1. Service Interface	21
2.5.2. Service Implementation	21
2.5.3. Service Registration	21
2.6. Unit Tests	21
2.6.1. XUnit	21
2.6.2. NUnit	22
2.6.3. FluentAssertions	23
2.6.4. Moq	24
2.6.5. Testen von Minimal APIs	25

1. Frontend

1.1. Authentication

1.1.1. Interceptor

Beschreibung: Der HTTP-Interceptor ist eine Middleware-Komponente, die automatisch bei **jedem** ausgehenden HTTP-Request ausgeführt wird. Er fügt den JWT-Token aus dem SessionStorage als Bearer-Token in den Authorization-Header ein, sodass authentifizierte API-Anfragen möglich sind.

Wichtige Hinweise:

- SessionStorage wird beim Schließen des Browser-Tabs gelöscht - für persistente Anmeldung localStorage verwenden
- Der Interceptor muss in `app.config.ts` unter `provideHttpClient(withInterceptors([authInterceptor]))` registriert werden

```
export const authInterceptor: HttpInterceptorFn = (request, next) => {
  const jwt = sessionStorage.getItem('jwt');
  const isLoggedIn = jwt;
  const isApiUrl = request.url.startsWith(environment.apiUrl);
  if (isLoggedIn && isApiUrl) {
    request = request.clone({
      headers: { Authorization: `Bearer ${jwt}` }
    });
  }

  return next(request);
}
```

1.1.2. Auth Guard

Beschreibung: Ein Route Guard, der den Zugriff auf geschützte Routen kontrolliert. Er prüft vor dem Laden einer Route, ob ein gültiger JWT-Token im SessionStorage vorhanden ist. Ist dies nicht der Fall, wird der User automatisch zur Login-Seite umgeleitet.

Wichtige Hinweise:

- Muss in den `app.routes.ts` bei den jeweiligen Routen mit `canActivate: [AuthGuard]` aktiviert werden
- Der Guard muss in `app.config.ts` mit `provideRouter(routes)` registriert werden

```
import { Router, CanActivate } from '@angular/router';
import { inject, Injectable } from '@angular/core';

@Injectable({
```

```

    providedIn: 'root'
  })
  export class AuthGuard implements CanActivate {
    private router = inject(Router);

    canActivate(): boolean {
      const jwt = sessionStorage.getItem('jwt');
      if (!jwt) {
        this.router.navigate(['login']);
        return false;
      }
      return true;
    }
  }
}

```

1.2. Environment

Beschreibung: Environment-Dateien ermöglichen unterschiedliche Konfigurationen für Umgebungen.

1.2.1. Environment Definition

```

// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5000/api'
};

```

1.2.2. Environment Usage

Beschreibung: Import der Environment-Konfiguration in Services oder Components.

```

import { environment } from '../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private apiUrl = environment.apiUrl;
}

```

1.3. Directives

1.3.1. Filter Directives

Beschreibung: Custom Validators für Template-Driven Forms. Diese Directive validiert Benutzereingaben **synchron** und gibt sofortiges Feedback. Ideal für einfache Validierungen wie Zahlenbereich, Format-Checks oder Pflichtfelder.

Wichtige Hinweise:

- Muss im `imports` Array der Komponente registriert werden (Standalone Components)
- Bei NgModule: Im `declarations` Array des Moduls registrieren
- Die Directive muss das `Validator` Interface implementieren
- Rückgabe `null` bedeutet valide, jedes andere Objekt ist ein Fehler
- `ValidationErrors` Objekt kann mehrere Fehler enthalten: `{ invalid: true, outOfRange: true }`
- Prüfe immer auf leere Werte (`!control.value || control.value === ""`) vor der Validierung
- Verwende sprechende Error-Keys: `{ hourOutOfRange: true }` statt `{ invalid: true }`
- Zeige Fehlermeldungen mit `@if(hourInput.errors?.['hourOutOfRange']) {<div>...</div>}`

```
@Directive({
  selector: '[appValidateHour]',
  providers: [
    {
      provide: NG_VALIDATORS,
      useExisting: forwardRef(() => NameDirective),
      multi: true
    }
  ]
})
export class ValidateHour implements Validator {
  validate(control: AbstractControl): ValidationErrors | null {
    if(control.value === "") return {errorMessage: true};
    return null;
  }
}
```

1.3.2. Async Directives

Beschreibung: Async Validators für komplexe Validierungen die eine Backend-Abfrage erfordern, z.B. Prüfung ob Username bereits existiert, ob eine ID gültig ist, oder ob ein Wert in der Datenbank vorhanden ist. Die Validierung läuft asynchron und blockiert das UI nicht.

Wichtige Hinweise:

- Muss `AsyncValidator` Interface implementieren (nicht `Validator`!)
- Provider-Konfiguration mit `NG_ASYNC_VALIDATORS` ist zwingend erforderlich
- `multi: true` erlaubt mehrere Async Validators auf einem Input

- Rückgabe muss `Observable<ValidationErrors | null>` sein
- Während der Validierung hat das Control den Status `PENDING`

Best Practice:

- Verwende `debounceTime(300)` um Requests zu verzögern bis User fertig getippt hat
- Zeige Loading-Spinner während `control.pending === true`
- Cache API-Responses um unnötige Requests zu vermeiden
- Kombiniere mit synchroner Validierung: erst Format prüfen, dann Backend-Call

```
@Directive({
  selector: '[appRomanCheck]',
  providers: [
    {
      provide: NG_ASYNC_VALIDATORS,
      useExisting: RomanCheck,
      multi: true
    }
  ]
})
export class RomanCheck implements AsyncValidator {
  private readonly dataService: DataService = inject(DataService);

  validate(control: AbstractControl): Observable<ValidationErrors | null> {
    const value: string = control.value;

    return from(this.dataService.isValid(value)).pipe(
      map((result: boolean) => (result ? null : { invalid: true }))
    );
  }
}
```

1.4. Routing

1.4.1. Routes

Beschreibung: Die zentrale Router-Konfiguration definiert alle verfügbaren Routen der Applikation. Jede Route mappt einen URL-Pfad auf eine Component und kann optional Guards, Resolver oder weitere Konfigurationen enthalten.

Wichtige Hinweise:

- Reihenfolge der Routes ist wichtig! Die erste passende Route wird verwendet
- `pathMatch: 'full'` bei Redirects verwenden, sonst matched jede URL die mit dem Pfad beginnt
- Optionale Wildcard-Route `{ path: '*', component: NotFoundComponent }` als *letzte Route, die als Fallback dient

```
export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'login'},
  { path: 'login', component: LoginComponent },
  { path: 'home', component: HomeComponent, canActivate: [AuthGuard]},
  { path: 'overview', component: InputDeviceOverviewComponent, canActivate:
[AuthGuard]}
];
```

1.4.2. Navigation

Beschreibung: Programmatische Navigation ermöglicht das Wechseln zwischen Routes im TypeScript-Code, z.B. nach erfolgreicher Formular-Submission, Login oder bei Button-Clicks. Der Router-Service stellt verschiedene Methoden für Navigation bereit.

Wichtige Hinweise:

- `router.navigate(['/path'])` ist relativ zum Root (absoluter Pfad)
- `router.navigate(['./path'], {relativeTo: route})` ist relativ zur aktuellen Route
- Query Params separat mit Options-Object übergeben

```
searchTerm = signal<string>('');

this.router.navigate(["/overview"], {
  queryParams: { searchTerm: this.searchTerm() }
});
```

1.4.3. Path Parameter

Beschreibung: Path Parameter (Route Parameters) sind dynamische Segmente in der URL, z.B. `/details/42` wobei `42` die ID ist. Sie werden in der Route-Konfiguration mit `:paramName` definiert und ermöglichen das Laden von spezifischen Ressourcen.

Wichtige Hinweise:

- Parameter sind **immer** strings! Konvertierung zu number erforderlich: `parseInt(id)`

```
// In app.routes.ts
export const routes: Routes = [
  { path: 'details/:id', component: InputDeviceDetails, canActivate: [AuthGuard]}
];

// Navigation
deviceId = signal<number>(42);
this.router.navigate(["/details", this.deviceId()]);

// In Component
deviceId = signal<number>(0);
```

```
const id: string | null = this.activeRoute.snapshot.paramMap.get("id");
this.deviceId.set(parseInt(id || "0"));
```

1.4.4. Query Parameter

Beschreibung: Query Parameter sind optionale Parameter die nach dem `?` in der URL stehen, z.B. `/search?term=angular&page=2`. Sie werden für Filterung, Sortierung, Pagination oder optionale Konfiguration verwendet und überleben Route-Changes.

Wichtige Hinweise:

- Query Params sind **immer** strings oder string arrays (`?id=1&id=2`)
- `snapshot.paramMap.get("id")` liefert Path Params, nicht Query Params!
- Für Query Params: `snapshot.queryParamMap.get("searchTerm")` verwenden

```
// Navigation
searchTerm = signal<string>('');

this.router.navigate(["/overview"], {
  queryParams: { searchTerm: this.searchTerm() }
});

// In Component
deviceId = signal<number>(0);

const id: string | null = this.activeRoute.snapshot.queryParamMap.get("id");
this.deviceId.set(parseInt(id || "0"));
```

1.5. Forms

1.5.1. Reactive Forms

Beschreibung: Reactive Forms (Model-Driven) sind die empfohlene Methode für komplexe Formulare in Angular. Das Form-Model wird im TypeScript-Code definiert und das Template bindet sich daran. Bietet volle Kontrolle, Testbarkeit und Type-Safety.

Wichtige Hinweise:

- `ReactiveFormsModule` muss importiert werden
- `[formGroup]` Directive auf `<form>` Element binden
- Verwende `FormBuilder` Service für sauberere Syntax

TypeScript:

```
loginForm: FormGroup;
```

```

constructor(private fb: FormBuilder) {
  this.loginForm = this.fb.group({
    username: ['', Validators.required],
    password: ['', Validators.required],
  });
}

onSubmit() {
  if (this.loginForm.valid) {
    const { username, password } = this.loginForm.value;
    // Handle login logic
  }
}

```

HTML:

```

<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <label for="username" name="username">Username:</label>
  <input id="username" type="text" formControlName="username" />
  <label for="password" name="password">Password:</label>
  <input id="password" type="password" formControlName="password" />
  <button type="submit" [disabled]="!loginForm.valid">Login</button>
</form>

```

1.5.2. Template Driven Forms

Beschreibung: Template Driven Forms sind einfacher zu implementieren und ähneln AngularJS Formularen. Die Form-Logik liegt hauptsächlich im Template mit Directives. Gut geeignet für einfache Formulare mit wenigen Feldern.

Wichtige Hinweise:

- `FormsModule` muss importiert werden
- `[(ngModel)]` bindet an Component-Property (Two-Way Binding)

TypeScript:

```

username = model<string>('');
password = model<string>('');

onSubmit() {
  // Handle login logic with this.username() and this.password()
}

```

HTML:

```

<form (ngSubmit)="onSubmit()" #loginForm="ngForm">
  <label for="username" name="username">Username:</label>
  <input
    id="username"
    type="text"
    name="username"
    required
    app-validation-directive
    [(ngModel)]="username"
  />
  <label for="password" name="password">Password:</label>
  <input
    id="password"
    type="password"
    name="password"
    required
    app-validation-directive
    [(ngModel)]="password"
  />
  <button type="submit" [disabled]="!loginForm.form.valid">Login</button>
</form>

```

1.6. Signals

Beschreibung: Signals sind Angulars neues Reaktivitäts-Primitiv (ab v16). Sie ermöglichen Fine-Grained Reactivity, Change Detection Optimierung und bessere Performance. Ein Signal ist ein Wrapper um einen Wert der automatisch Dependencies trackt.

Wichtige Hinweise:

- `signal()` erstellt writable Signal, `computed()` für abgeleitete Werte
- Signals müssen mit `()` aufgerufen werden um den Wert zu lesen: `count()`
- Zum Schreiben: `count.set(5)` oder `count.update(v ⇒ v + 1)`

1.6.1. Models

Beschreibung: Two-Way Binding von Signals in Standalone Components. Das `model<T>()` Decorator erstellt ein Signal-Property das automatisch mit dem Template synchronisiert wird. Änderungen im Template aktualisieren das Signal und umgekehrt.

Wichtige Hinweise:

- `model<T>(defaultValue)` initialisiert das Signal mit einem Standardwert
- Im Template mit `[(ngModel)]="propertyName"` binden
- Änderungen im Template aktualisieren automatisch das Signal

Child Component (TypeScript):

```
deviceName = model<string>>('');  
deviceDescription = model<string>>('');
```

Child Component (HTML):

```
<form>  
  <label for="deviceName" name="deviceName">Device Name:</label>  
  <input id="deviceName" type="text" [(ngModel)]="deviceName" />  
  <label for="deviceDescription" name="deviceDescription">Description:</label>  
  <input id="deviceDescription" type="text" [(ngModel)]="deviceDescription" />  
</form>
```

Parent Component (TypeScript):

```
name = signal<string>('Initial Device');  
description = signal<string>('Initial Description');
```

Parent Component (HTML):

```
<app-device-form  
  [(deviceName)]="name"  
  [(deviceDescription)]="description"  
>
```

1.6.2. Input

Beschreibung: Signal-basierte Inputs ersetzen das traditionelle `@Input()` Decorator und bieten Type-Safety, automatische Change Detection und bessere Integration mit dem Signal-System. Parent Components können reaktiv Daten an Child Components übergeben.

Wichtige Hinweise:

- `input.required<T>()` wirft Compile-Error wenn Property nicht gesetzt wird
- `input<T>(defaultValue)` für optionale Inputs mit Fallback-Wert
- Signal Inputs sind **readonly** - Child kann Wert nicht direkt ändern

Child Component (TypeScript):

```
device = input.required<InputDevice>();  
isRequired = input<boolean>(false);  
title = input<string>('Default Title');
```

Parent Component (HTML):

```
<app-device-card
  [device]="selectedDevice()"
  [isRequired]="true"
  [title]="'Device Details'"
/>
```

Parent Component (TypeScript):

```
selectedDevice = signal<InputDevice>({ id: 1, name: 'Device 1' });
```

1.6.3. Output

Beschreibung: Signal-basierte Outputs ersetzen `@Output()` EventEmitter und ermöglichen typsichere Event-Kommunikation von Child zu Parent Component. Events werden mit `output<T>()` definiert und können mit oder ohne Daten emittet werden.

Wichtige Hinweise:

- `output<T>()` erstellt einen Event-Emitter, **kein** Signal!
- Events werden mit `.emit(value)` ausgelöst (wie bei EventEmitter)
- Parent muss Event im Template mit `(eventName)="handler($event)"` abonnieren

Child Component (TypeScript):

```
deviceSaved = output<InputDevice>();
cancelled = output<void>();

handleSave() {
  const device: InputDevice = { id: 1, name: 'Device 1' };
  this.deviceSaved.emit(device);
}

handleCancel() {
  this.cancelled.emit();
}
```

Child Component (HTML):

```
<button (click)="handleSave()">Save</button>
<button (click)="handleCancel()">Cancel</button>
```

Parent Component (TypeScript):

```
onDeviceSaved(device: InputDevice) {
```

```

    console.log('Device saved:', device);
  }

  onCancelled() {
    console.log('Form cancelled');
  }

```

Parent Component (HTML):

```

<app-device-form
  (deviceSaved)="onDeviceSaved($event)"
  (cancelled)="onCancelled()"
/>

```

1.7. HTTP Client

Beschreibung: Der HttpClient-Service ist Angulars zentrale API für HTTP-Kommunikation. Er bietet eine Observable-basierte API für REST-Calls, automatische JSON-Parsing, Request/Response-Interceptors und Type-Safety durch Generics.

Wichtige Hinweise:

- `HttpClientModule` muss in `app.config.ts` mit `provideHttpClient()` registriert werden
- Observables sind lazy - ohne `.subscribe()` wird kein Request gesendet
- Error Handling mit `.pipe(catchError())` implementieren

```

import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders, HttpParams } from '@angular/common/http';
import { Observable, observeOn } from 'rxjs';
import { Game } from '../models/game.model';

const httpOptions = {
  headers: new HttpHeaders({
    'Content-Type': 'application/json'
  })
}

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private apiUrl = "https://h-aitenbichler.cloud.htl-leonding.ac.at/restserver";

  public readonly header = new HttpHeaders({
    'Content-Type': 'application/json'
  });

  constructor(private http: HttpClient) { }

```

```
// Optional - Hilfsmethode für Arrays als Query Parameter
private toQueryParamArray(content: string[]): string {
    return "?sudoku=" + content.join("&sudoku=");
}
}
```

1.8. Observables

HttpClient liefert immer Observable<T> zurück. Ausgeführt wird es erst mit subscribe().

1.8.1. subscribe mit next + error

```
this.service.getData().subscribe({
  next: data => {
    console.log('OK:', data);
  },
  error: err => {
    console.error('Fehler:', err);
  }
});
```

1.8.2. Eigenes Observable (Minimalbeispiel)

```
const obs$ = new Observable(o => {
  o.next('Hallo');
  o.error('Fehler');
});

obs$.subscribe({
  next: v => console.log(v),
  error: e => console.error(e)
});
```

1.9. HTTP Requests

1.9.1. GET

```
getGames(): Observable<Game[]> {
  return this.http.get<Game[]>(`${this.apiUrl}/game`);
}
```

```
getGame(id: number): Observable<Game> {
  return this.http.get<Game>(`${this.apiUrl}/game/${id}`);
}
```

```
}
```

1.9.2. POST

```
postGame(game: Game): Observable<Game> {  
    return this.http.post<Game>(`${this.apiUrl}/game`, game);  
}
```

1.9.3. PUT

```
putGame(id: number, game: Game) {  
    return this.http.put(`${this.apiUrl}/game/${id}`, game,  
        {  
            headers: this.header,  
            observe: 'response'  
        });  
}
```

1.9.4. PATCH

```
patchGame(id: number, partialGame: Partial<Game>) {  
    return this.http.patch(`${this.apiUrl}/game/${id}`, partialGame,  
        {  
            headers: this.header,  
            observe: 'response'  
        });  
}
```

1.9.5. DELETE

```
deleteGame(id: number) {  
    return this.http.delete(`${this.apiUrl}/game/${id}`, {  
        headers: this.header,  
        observe: 'response'  
    });  
}
```

1.9.6. Request Parameters

Beschreibung: HTTP Request Parameters ermöglichen das Übergeben von Daten an API-Endpoints über drei Hauptmethoden: Query Parameters in der URL, Path Parameters als Teil der URL, und Request Body für komplexe Daten.

Query Parameters

Beschreibung: Query Parameters werden als Key-Value-Pairs an die URL angehängt (`?key=value&key2=value2`).

```
getDevices(search: string, page: number): Observable<Device[]> {
  const params = new HttpParams()
    .set('search', search)
    .set('page', page.toString());
  return this.http.get<Device[]>(`${this.apiUrl}/devices`, { params });
}
```

Path Parameters

Beschreibung: Path Parameters werden direkt in die URL als Segmente eingebaut (`/api/devices/123`).

```
getDevice(id: number): Observable<Device> {
  return this.http.get<Device>(`${this.apiUrl}/devices/${id}`);
}
```

Request Body

Beschreibung: Request Body überträgt komplexe Daten als JSON bei POST, PUT und PATCH Requests.

```
createDevice(device: Device): Observable<Device> {
  return this.http.post<Device>(`${this.apiUrl}/devices`, device);
}

updateDevice(id: number, device: Device): Observable<Device> {
  return this.http.put<Device>(`${this.apiUrl}/devices/${id}`, device);
}
```

1.10. Template Syntax

1.10.1. @for

Beschreibung: Die `@for` Direktive ermöglicht das Iterieren über Arrays oder Collections im Template. Sie ist eine Alternative zu `*ngFor` und bietet eine klarere Syntax für das Durchlaufen von Listen.

Wichtige Hinweise:

- Verwende `@for (item of items; track item)` um über ein Array zu iterieren
- `track item` hilft Angular, Elemente effizient zu rendern und zu aktualisieren

```
@for (device of devices(); track device.id; let i = $index) {
  <div>{{ i }}: {{ device.name }}</div>
}
```

1.10.2. @if

Beschreibung: Die `@if` Direktive ermöglicht bedingtes Rendern von Template-Abschnitten basierend auf einem boolean Ausdruck. Sie ist eine Alternative zu `*ngIf` und bietet eine klarere Syntax für bedingte Logik.

Wichtige Hinweise:

- Verwende `@if (condition) { ... }` um einen Block nur bei wahrer Bedingung zu rendern
- Optional kann ein `@else { ... }` Block hinzugefügt werden

```
@if (isLoggedIn()) {
  <div>Welcome back, user!</div>
} @else {
  <div>Please log in to continue.</div>
}
```

1.11. Table

```
<table>
  <thead>
    <tr>
      <th>Device</th>
      <th>Description</th>
      <th>Reservation</th>
      <th></th>
    </tr>
  </thead>
  @for (device of filteredDevices(); track device.id) {
    <tbody>
      <tr class="border-bottom">
        <td>{{ device.name }}</td>
        <td>{{ device.description }}</td>
        <td>{{ formatReservationDates(device.reservations) }}</td>
        <td>
          <button class="btn btn-secondary" (click)="detailInputDevice(device.id)">
            Reserve
          </button>
        </td>
      </tr>
    </tbody>
  }
}
```

```
</table>
```

2. Backend

2.1. CDI (Dependency Injection)

Beschreibung: Dependency Injection Container registriert Services die dann automatisch in Controllern/Endpoints injiziert werden können. Ermöglicht loose coupling, bessere Testbarkeit und zentrale Konfiguration.

Wichtige Hinweise:

- Services müssen **vor** `var app = builder.Build()` registriert werden
- **AddTransient**: neue Instanz bei jeder Injection (stateless Services)
- **AddScoped**: eine Instanz pro HTTP-Request (z.B. DbContext)
- **AddSingleton**: eine Instanz für gesamte App-Lifetime (Caching, Configuration)
- Interface-Registration (`<IService, Service>`) erlaubt Mocking in Tests

Füge Service in Backend hinzu:

```
builder.Services.AddTransient<IMessageService, MessageService>();
```

```
builder.Services.AddTransient<MessageService>();
```

2.2. Swagger

2.2.1. Swagger Builder

Beschreibung: Swagger/OpenAPI UI generiert automatisch interaktive API-Dokumentation. Endpoints können direkt getestet werden, Request/Response Schemas werden visualisiert. Essential für Frontend-Entwickler und API-Testing.

Wichtige Hinweise:

- **AddEndpointsApiExplorer()** muss **vor** **AddSwaggerGen()** stehen

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddSingleton<IRomanNumerals, RomanNumerals>();

// Add cors here if needed
builder.Services.AddCors(options =>
{
```

```

options.AddPolicy("AllowAngular",
    policy => policy
        .WithOrigins("http://localhost:4200")
        .AllowAnyHeader()
        .AllowAnyMethod());
});

var app = builder.Build();

// Add cors here if needed
app.UseCors("AllowAngular");

app.UseSwagger();
app.UseSwaggerUI();

```

2.2.2. Cors

Beschreibung: CORS (Cross-Origin Resource Sharing) erlaubt Browser Requests von anderen Domains (z.B. Frontend auf localhost:4200 zu API auf localhost:5000). Ohne CORS blockiert Browser die Requests aus Sicherheitsgründen.

Wichtige Hinweise:

- CORS-Policy muss **vor** `app.Build()` registriert werden
- `UseCors()` muss **vor** `UseAuthorization()` stehen (Middleware-Reihenfolge!)

```

// Reihenfolge siehe oben
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngular",
        policy => policy
            .WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

app.UseCors("AllowAngular");

```

2.3. Minimal API

2.3.1. GET

```

app.MapGet("/isValid", (string literal, IRomanNumerals service) =>
{
    try
    {
        service.ConvertFromRomanLiteral(literal);
    }
}

```

```

        return true;
    }
    catch (Exception ex) when (ex is ArgumentOutOfRangeException or ArgumentException)
    {
        return false;
    }
});

```

2.3.2. POST

```

app.MapPost("/devices", (InputDevice device, IDeviceService service) =>
{
    try
    {
        var created = service.CreateDevice(device);
        return Results.Created($"/devices/{created.Id}", created);
    }
    catch (Exception ex)
    {
        return Results.BadRequest(ex.Message);
    }
});

```

2.3.3. PUT

```

app.MapPut("/devices/{id}", (int id, InputDevice device, IDeviceService service) =>
{
    try
    {
        var updated = service.UpdateDevice(id, device);
        return updated != null ? Results.Ok(updated) : Results.NotFound();
    }
    catch (Exception ex)
    {
        return Results.BadRequest(ex.Message);
    }
});

```

2.3.4. PATCH

```

app.MapPatch("/devices/{id}/status", (int id, string status, IDeviceService service)
=>
{
    try
    {
        var updated = service.UpdateDeviceStatus(id, status);
    }
});

```

```

        return updated ? Results.NoContent() : Results.NotFound();
    }
    catch (Exception ex)
    {
        return Results.BadRequest(ex.Message);
    }
});

```

2.3.5. DELETE

```

app.MapDelete("/devices/{id}", (int id, IDeviceService service) =>
{
    try
    {
        var deleted = service.DeleteDevice(id);
        return deleted ? Results.NoContent() : Results.NotFound();
    }
    catch (Exception ex)
    {
        return Results.BadRequest(ex.Message);
    }
});

```

2.4. Authentication

Beschreibung: JWT (JSON Web Token) Authentication für stateless API-Authentifizierung. Token enthält Claims (User-ID, Roles) und wird bei jedem Request im Authorization-Header gesendet.

```

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
builder.Configuration
["Jwt:Key"]))
        };
    });

```

2.5. Services

Beschreibung: Services kapseln Business-Logic und werden via Dependency Injection in Endpoints/Controller injiziert. Ermöglicht Separation of Concerns, bessere Testbarkeit und Code-Reuse.

Wichtige Hinweise:

- Services müssen in `Program.cs` registriert werden (siehe 2.1/2.5.3)

2.5.1. Service Interface

```
namespace YourProject.Services;

public interface IYourService
{
    // Definiere deine Service-Methoden hier
}
```

2.5.2. Service Implementation

```
namespace YourProject.Services;

public class YourService : IYourService
{
    // Implementiere deine Service-Methoden hier
}
```

2.5.3. Service Registration

```
builder.Services.AddTransient<IYourService, YourService>();
// oder
builder.Services.AddTransient<YourService>();
```

2.6. Unit Tests

2.6.1. XUnit

Beschreibung: XUnit ist ein populäres Testing-Framework für .NET. Es ermöglicht das Schreiben von Unit-Tests, Integrationstests und End-to-End-Tests. Tests werden in separaten Projekten organisiert und können automatisiert ausgeführt werden.

Wichtige Hinweise:

- Test-Projekt muss `xunit` und `xunit.runner.visualstudio` NuGet-Pakete referenzieren

- Test-Klassen müssen mit `[Fact]` oder `[Theory]` dekoriert werden

```
using Xunit;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private readonly IYourService _yourService;

    public YourServiceTests()
    {
        _yourService = new YourService();
    }

    [Fact]
    public void YourMethod_ShouldReturnExpectedResult()
    {
        // Arrange
        var input = ...;

        // Act
        var result = _yourService.YourMethod(input);

        // Assert
        Assert.Equal(expectedResult, result);
    }
}
```

2.6.2. NUnit

Beschreibung: NUnit ist ein weiteres weit verbreitetes Testing-Framework für .NET. Es bietet ähnliche Funktionalitäten wie XUnit, mit einer etwas anderen Syntax und Struktur. NUnit unterstützt auch Data-Driven Tests und bietet umfangreiche Assertions.

Wichtige Hinweise:

- Test-Projekt muss `NUnit` und `NUnit3TestAdapter` NuGet-Pakete referenzieren

```
using NUnit.Framework;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private IYourService _yourService;

    [SetUp]
    public void Setup()
```

```

{
    _yourService = new YourService();
}

[Test]
public void YourMethod_ShouldReturnExpectedResult()
{
    // Arrange
    var input = ...;

    // Act
    var result = _yourService.YourMethod(input);

    // Assert
    Assert.AreEqual(expectedResult, result);
}
}

```

2.6.3. FluentAssertions

Beschreibung: FluentAssertions ist eine Erweiterung für Testing-Frameworks wie XUnit und NUnit, die eine lesbare und ausdrucksstarke Syntax für Assertions bietet. Es verbessert die Klarheit der Tests und erleichtert das Debugging.

Wichtige Hinweise:

- Test-Projekt muss **FluentAssertions** NuGet-Paket referenzieren

```

using FluentAssertions;
using Xunit;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private readonly IYourService _yourService;

    public YourServiceTests()
    {
        _yourService = new YourService();
    }

    [Fact]
    public void YourMethod_ShouldReturnExpectedResult()
    {
        // Arrange
        var input = ...;

        // Act
        var result = _yourService.YourMethod(input);
    }
}

```

```

        // Assert
        result.Should().Be(expectedResult);
    }
}

```

2.6.4. Moq

Beschreibung: Moq ist ein Mocking-Framework für .NET, das es ermöglicht, Abhängigkeiten zu mocken und zu stubben. Ideal für Unit-Tests, um isolierte Testszenarien zu erstellen.

Wichtige Hinweise:

- Test-Projekt muss **Moq** NuGet-Paket referenzieren
- Verwende **Mock<T>** um Mocks zu erstellen und Verhalten zu konfigurieren

```

using Moq;
using Xunit;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private readonly Mock<IDependencyService> _dependencyServiceMock;
    private readonly IYourService _yourService;

    public YourServiceTests()
    {
        _dependencyServiceMock = new Mock<IDependencyService>();
        _yourService = new YourService(_dependencyServiceMock.Object);
    }

    [Fact]
    public void YourMethod_ShouldReturnExpectedResult()
    {
        // Arrange
        var input = ...;
        _dependencyServiceMock.Setup(ds => ds.SomeMethod(It.IsAny<Type>()))
            .Returns(expectedValue);

        // Act
        var result = _yourService.YourMethod(input);

        // Assert
        Assert.Equal(expectedResult, result);
        _dependencyServiceMock.Verify(ds => ds.SomeMethod(It.IsAny<Type>()), Times
.Once);
    }
}

```

```
}
```

2.6.5. Testen von Minimal APIs

Beschreibung: Minimal APIs können mit dem `WebApplicationFactory<TEntryPoint>` aus dem `Microsoft.AspNetCore.Mvc.Testing` Paket getestet werden. Dies ermöglicht das Starten der API in einem Test-Host und das Senden von HTTP-Requests.

Wichtige Hinweise:

- Test-Projekt muss `Microsoft.AspNetCore.Mvc.Testing` NuGet-Paket referenzieren
- Verwende `HttpClient` um Requests an die API zu senden

```
using Microsoft.AspNetCore.Mvc.Testing;
using System.Net.Http;
using System.Threading.Tasks;
using Xunit;
namespace YourProject.Tests;

public class YourApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public YourApiTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetEndpoint_ShouldReturnSuccessStatusCode()
    {
        // Act
        var response = await _client.GetAsync("/your-endpoint");

        // Assert
        response.EnsureSuccessStatusCode();
        var content = await response.Content.ReadAsStringAsync();
        Assert.NotNull(content);
    }
}
```